

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук
Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 9

дисциплина: Архитектура компьютера

Студент: Томилова Валентина
Станиславовна

Группа: НКАбд-06-25

МОСКВА

2025 г.

Оглавление

<u>1 Цель работы</u>	3
<u>2 Задания</u>	4
<u>3 Теоретическое введение</u>	5
<u>4 Выполнение работы</u>	6
<u>Реализация подпрограмм в NASM</u>	6
<u>Отладка программ с помощью GDB</u>	10
<u>Обработка аргументов командной строки в GDB</u>	28
<u>Задание для самостоятельной работы</u>	30
<u>5 Выводы</u>	40
<u>Список литературы</u>	41

Рисунок 0.1(создание каталога)	8
Рисунок 0.2(создание файла)	8
Рисунок 0.3(текст программы)	9
Рисунок 0.4(создание исполняемого файла)	9
Рисунок 0.5(создание исполняемого файла)	9
Рисунок 0.6(запуск исполняемого файла)	9
Рисунок 0.7(проверка работы программы)	10
Рисунок 0.8 (программа с добавлением подпрограммы)	11
Рисунок 0.9 (создание исполняемого файла)	12
Рисунок 0.10 (создание исполняемого файла)	12
Рисунок 0.11 (запуск исполняемого файла)	12
Рисунок 0.12 (проверка работы программы)	12
Рисунок 0.13 (создание файла)	12
Рисунок 0.14 (текст программы)	13
Рисунок 0.15 (создание исполняемого файла)	13
Рисунок 0.16 (создание исполняемого файла)	13
Рисунок 0.17 (Загрузка исполняемого файла в отладчик gdb)	14
Рисунок 0.18(запуск программы при помощи gup)	14
Рисунок 0.19(установка брейкпоинта)	14
Рисунок 0.20(запуск программы)	15
Рисунок 0.21(просмотр дисассимилированного кода программы)	15
Рисунок 0.22(переключение на отображение команд с Intel'овским синтаксисом)	16
Рисунок 0.23 (layout asm)	16
Рисунок 0.24 (layout regs)	17
Рисунок 0.25 (info breakpoints)	17
Рисунок 0.26(установка точки останова)	18
Рисунок 0.27(просмотр информации о установленных точках)	18
Рисунок 0.28 (просмотр содержимого регистров)	19
Рисунок 0.29(stepi)	20
Рисунок 0.30(info registers)	21
Рисунок 0.31(stepi)	21
Рисунок 0.32(info registers)	22
Рисунок 0.33(stepi)	23
Рисунок 0.34(info registers)	23
Рисунок 0.35(stepi)	24
Рисунок 0.36(info registers)	25
Рисунок 0.37(stepi)	26
Рисунок 0.38(info registers)	27
Рисунок 0.39(значение переменной msg1)	27
Рисунок 0.40(значение переменной msg2)	28
Рисунок 0.41(изменение первого символа msg1)	28
Рисунок 0.42 (изменение символа msg2)	28
Рисунок 0.43(значение регистра edx в разных форматах)	29
Рисунок 0.44 (изменение значения регистра ebx)	29
Рисунок 0.45(завершение выполнения программы)	29
Рисунок 0.46(Создание копии файла)	30
Рисунок 0.47(создание исполняемого файла)	30
Рисунок 0.48(загрузка исполняемого файла в отладчик)	30
Рисунок 0.49(установка точки перед первой инструкцией)	31
Рисунок 0.50(запуск)	31
Рисунок 0.51(Просмотр остальных позиций стека)	31
Рисунок 0.52 (создание копии файла)	32
Рисунок 0.53(текст программы)	33
Рисунок 0.54(проверка работы программы)	34
Рисунок 0.55(создание файла)	34
Рисунок 0.56(текст программы)	35
Рисунок 0.57(создание исполняемого файла)	35
Рисунок 0.58(загрузка исполняемого файла в отладчик)	36
Рисунок 0.59(установка брейкпоинта)	36
Рисунок 0.60(запуск программы)	36
Рисунок 0.61(Результат до исправления кода)	36

Рисунок 0.62(layout regs)	37
Рисунок 0.63(первый step)	37
Рисунок 0.64(второй step)	38
Рисунок 0.65(третий step)	39
Рисунок 0.66(step находим ошибку)	40
Рисунок 0.67(исправленный код)	41
Рисунок 0.68(результат после исправления)	41

1 Цель работы

Приобретение навыков написания программ с использованием подпрограмм.
Знакомство с методами отладки при помощи GDB и его основными возможностями

2 Задания

Реализация подпрограмм в NASM, Отладка программ с помощью GDB,
Задание для самостоятельной работы

3 Теоретическое введение

Отладка — процесс поиска и исправления ошибок в программе. Как правило, он состоит из четырех основных этапов:

1. **Обнаружение ошибки** — понимание того, что программа работает некорректно.
2. **Поиск места ошибки** — локализация фрагмента кода, вызывающего проблему.
3. **Определение причины** — анализ, почему в данном месте возникает ошибка.
4. **Исправление ошибки** — изменение кода для устранения проблемы, после чего цикл может повториться для других ошибок.

Можно выделить три основных типа ошибок:

- **Синтаксические ошибки** — нарушение правил языка программирования; обнаруживаются на этапе трансляции (компиляции/интерпретации).
- **Семантические (логические) ошибки** — программа запускается и выполняется, но выдает непредусмотренный или неверный результат из-за ошибочной логики.
- **Ошибки в процессе выполнения** — возникают во время работы программы и могут приводить к её аварийному завершению (например, деление на ноль или переполнение памяти).

Наиболее сложным часто оказывается второй этап — поиск места ошибки. Для этого полезно разбивать программу на части и проверять их отдельно, используя методы отладки (например, вывод промежуточных значений, пошаговое выполнение, тестирование модулей).

После локализации ошибки обычно проще понять её причину и внести необходимые исправления. Однако после исправления одной ошибки могут обнаруживаться другие, и процесс отладки продолжается.

4 Выполнение работы

Реализация подпрограмм в NASM

Создадим каталог для выполнения лабораторной работы №9, перейдем в него и создадим файл lab09-1.asm:

```
vstomilova@dk3n55 ~ $ mkdir ~/work/arch-pc/lab09  
vstomilova@dk3n55 ~ $ cd ~/work/arch-pc/lab09
```

Рисунок 0.1(создание каталога)

```
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ touch lab09-1.asm  
vstomilova@dk3n55 ~/work/arch-pc/lab09 $
```

Рисунок 0.2(создание файла)

В качестве примера рассмотрим программу вычисления арифметического выражения $\square(\square) = 2\square + 7$ с помощью подпрограммы `_calcul`. В данном примере $\square$ вводится с клавиатуры, а само выражение вычисляется в подпрограмме. Внимательно изучим текст программы и перепишем его


```

1 %include 'in_out.asm'
2
3 SECTION .data
4 msg: DB 'Введите x: ',0
5 result: DB '2x+7=',0
6 SECTION .bss
7 x: RESB 80
8 res: RESB 80
9 SECTION .text
10 GLOBAL _start
11 _start:
12
13 mov eax, msg
14 call sprint
15 mov ecx, x
16 mov edx, 80
17 call sread
18 mov eax, x
19 call atoi
20 call _calcul ; Вызов подпрограммы _calcul
21 mov eax, result
22 call sprint
23 mov eax, [res]
24 call iprintLF
25 call quit
26
27 ; Подпрограмма вычисления
28 ; выражения "2x+7"
29 _calcul:
30 mov ebx, 2
31 mul ebx
32 add eax, 7
33
34 mov [res], eax
35 ret

```

Рисунок 0.3(текст программы)

Создадим исполняемый файл и проверим его работу

```

vstomilova@dk3n55 ~/work/arch-pc/lab09 $ nasm -f elf lab09-1.asm

```

Рисунок 0.4(создание исполняемого файла)

```

vstomilova@dk3n55 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-1 lab09-1.o
vstomilova@dk3n55 ~/work/arch-pc/lab09 $

```

Рисунок 0.5(создание исполняемого файла)

```

vstomilova@dk3n55 ~/work/arch-pc/lab09 $ ./lab09-1
Введите x:

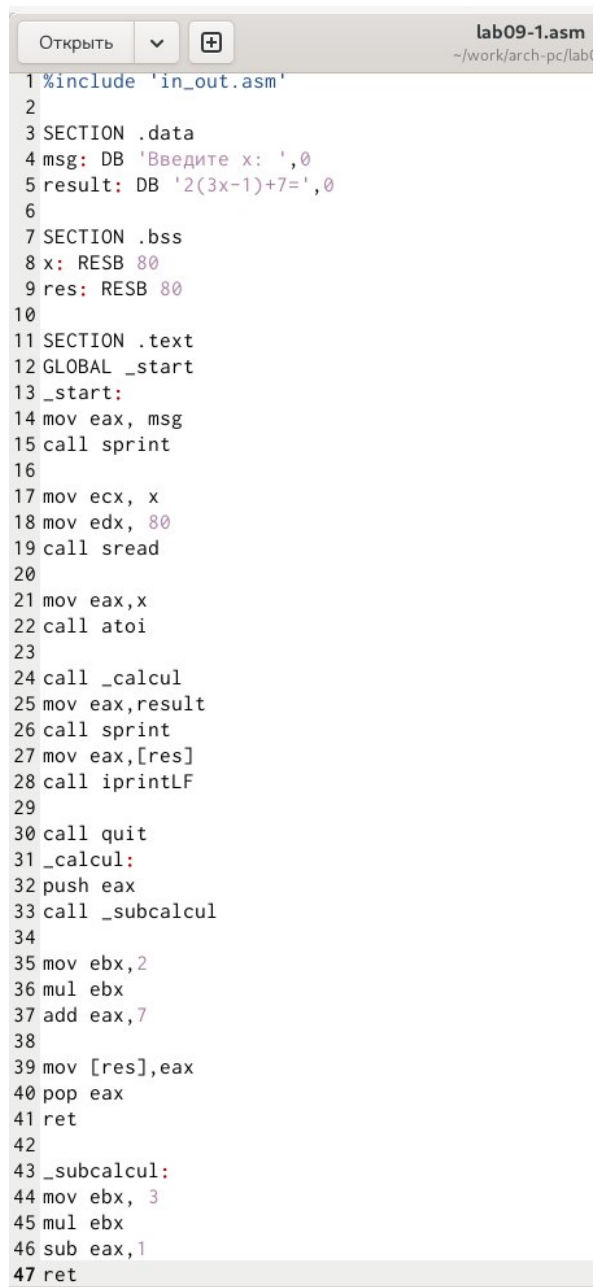
```

Рисунок 0.6(запуск исполняемого файла)

```
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ ./lab09-1
Введите x: 10
2x+7=27
vstomilova@dk3n55 ~/work/arch-pc/lab09 $
```

Рисунок 0.7(проверка работы программы)

Изменим текст программы, добавив подпрограмму `_subcalcul` в подпрограмму `_calcul`, для вычисления выражения $\square(\square(\square))$, где $\square$ вводится с клавиатуры, $\square(\square) = 2\square + 7$, $\square(\square) = 3\square - 1$. Т.е. $\square$ передается в подпрограмму `_calcul` и в нее в подпрограмму `_subcalcul`, где вычисляется выражение $\square(\square)$, результат возвращается в `_calcul` и вычисляется выражение $\square(\square(\square))$. Результат возвращается в основную программу для вывода результата на экран.



```
1 %include 'in_out.asm'
2
3 SECTION .data
4 msg: DB 'Введите x: ',0
5 result: DB '2(3x-1)+7=',0
6
7 SECTION .bss
8 x: RESB 80
9 res: RESB 80
10
11 SECTION .text
12 GLOBAL _start
13 _start:
14 mov eax, msg
15 call sprint
16
17 mov ecx, x
18 mov edx, 80
19 call sread
20
21 mov eax, x
22 call atoi
23
24 call _calcul
25 mov eax, result
26 call sprint
27 mov eax, [res]
28 call iprintLF
29
30 call quit
31 _calcul:
32 push eax
33 call _subcalcul
34
35 mov ebx, 2
36 mul ebx
37 add eax, 7
38
39 mov [res], eax
40 pop eax
41 ret
42
43 _subcalcul:
44 mov ebx, 3
45 mul ebx
46 sub eax, 1
47 ret
```

Рисунок 0.8 (программа с добавлением подпрограммы)

Создадим исполняемый файл и проверим его работу

```
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ nasm -f elf lab09-1.asm
```

Рисунок 0.9 (создание исполняемого файла)

```
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-1 lab09-1.o  
vstomilova@dk3n55 ~/work/arch-pc/lab09 $
```

Рисунок 0.10 (создание исполняемого файла)

```
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ ./lab09-1  
Введите x:
```

Рисунок 0.11 (запуск исполняемого файла)

```
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ ./lab09-1  
Введите x: 10  
2(3x-1)+7=65  
vstomilova@dk3n55 ~/work/arch-pc/lab09 $
```

Рисунок 0.12 (проверка работы программы)

Отладка программ с помощью GDB

Создадим файл lab09-2.asm с текстом программы из Листинга

```
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ touch lab09-2.asm  
vstomilova@dk3n55 ~/work/arch-pc/lab09 $
```

Рисунок 0.13 (создание файла)



```
1 SECTION .data
2 msg1: db "Hello, ",0x0
3 msg1Len: equ $ - msg1
4 msg2: db "world!",0xa
5 msg2Len: equ $ - msg2
6
7 SECTION .text
8 global _start
9
10 _start:
11 mov eax, 4
12 mov ebx, 1
13 mov ecx, msg1
14 mov edx, msg1Len
15 int 0x80
16
17 mov eax, 4
18 mov ebx, 1
19 mov ecx, msg2
20 mov edx, msg2Len
21 int 0x80
22
23 mov eax, 1
24 mov ebx, 0
25 int 0x80
```

Рисунок 0.14 (текст программы)

Получим исполняемый файл

```
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ nasm -f elf -g -l lab09-2.lst lab09-2.asm
```

Рисунок 0.15 (создание исполняемого файла)

```
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ nasm -f elf -g -l lab09-2.lst lab09-2.asm
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-2 lab09-2.o
```

Рисунок 0.16 (создание исполняемого файла)

Загрузим исполняемый файл в отладчик gdb

```

vstomilova@dk3n55 ~/work/arch-pc/lab09 $ gdb lab09-2
Exception caught while booting Guile.
Error in function "load-thunk-from-memory":
missing DT_GUILE_ENTRY
gdb: warning: Could not complete Guile gdb module initialization from:
/usr/share/gdb/guile/gdb/boot.scm.
Limited Guile support is available.
Suggest passing --data-directory=/path/to/gdb/data-directory.
GNU gdb (Gentoo 16.3 vanilla) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://bugs.gentoo.org/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-2...

```

Рисунок 0.17 (Загрузка исполняемого файла в отладчик gdb)

Проверьте работу программы, запустив ее в оболочке GDB с помощью команды run

```

(gdb) run
Starting program: /afs/.dk.sci.pfu.edu.ru/home/v/s/vstomilova/work/arch-pc/lab09/lab09-2
Hello, world!
[Inferior 1 (process 8081) exited normally]

```

Рисунок 0.18(запуск программы при помощи run)

Для более подробного анализа программы установим брейкпоинт на метку _start, с которой начинается выполнение любой ассемблерной программы, и запустим её

```

(gdb) break _start
Breakpoint 1 at 0x8049000: file lab09-2.asm, line 11.

```

Рисунок 0.19(установка брейкпоинта)

```
(gdb) run
Starting program: /afs/.dk.sci.pfu.edu.ru/home/v/s/vstomilova/work/arch-pc/lab09/lab09-2

Breakpoint 1, _start () at lab09-2.asm:11
11      mov eax, 4
```

Рисунок 0.20(запуск программы)

Посмотрим дисассимилированный код программы с помощью команды `disassemble` начиная с метки `_start`

```
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     $0x4,%eax
      0x08049005 <+5>:      mov     $0x1,%ebx
      0x08049009 <+10>:     mov     $0x004a0000,%ecx
      0x0804900f <+15>:     mov     $0x8,%edx
      0x08049014 <+20>:     int     $0x80
      0x08049016 <+22>:     mov     $0x4,%eax
      0x0804901b <+27>:     mov     $0x1,%ebx
      0x08049020 <+32>:     mov     $0x004a0000,%ecx
      0x08049025 <+37>:     mov     $0x7,%edx
      0x0804902a <+42>:     int     $0x80
      0x0804902c <+44>:     mov     $0x1,%eax
      0x08049031 <+49>:     mov     $0x0,%ebx
      0x08049036 <+54>:     int     $0x80
End of assembler dump.
```

Рисунок 0.21(просмотр дисассимилированного кода программы)

Переключимся на отображение команд с Intel'овским синтаксисом, введя команду `set disassembly-flavor intel`


```

(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     eax,0x4
    0x08049005 <+5>:      mov     ebx,0x1
    0x0804900a <+10>:     mov     ecx,0x804a000
    0x0804900f <+15>:     mov     edx,0x8
    0x08049014 <+20>:     int     0x80
    0x08049016 <+22>:     mov     eax,0x4
    0x0804901b <+27>:     mov     ebx,0x1
    0x08049020 <+32>:     mov     ecx,0x804a000
    0x08049025 <+37>:     mov     edx,0x7
    0x0804902a <+42>:     int     0x80
    0x0804902c <+44>:     mov     eax,0x1
    0x08049031 <+49>:     mov     ebx,0x0
    0x08049036 <+54>:     int     0x80
End of assembler dump.
(gdb) █

```

Рисунок 0.22(переключение на отображение команд с Intel'овским синтаксисом)

Перечислим различия отображения синтаксиса машинных команд в режимах АТТ и Intel. Включим режим псевдографики для более удобного анализа программы


```
Терминал - vstomilova@dk3n55 - lab09
Файл Правка Вид Терминал Вкладки Справка

Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffc2a0 0xffffc2a0

B+>0x8049000 <_start>    mov    eax,0x4
    0x8049005 <_start+5>  mov    ebx,0x1
    0x804900a <_start+10> mov    ecx,0x804a000
    0x804900f <_start+15> mov    edx,0x8
    0x8049014 <_start+20> int     0x80
    0x8049018 <_start+22> mov    eax,0x4

native process 8143 (asm) In: _start      L11  PC: 0x8049000
(gdb) layout regs
(gdb) 
```

Рисунок 0.23 (layout asm)

```
Терминал - vstomilova@dk3n55 - lab09
Файл  Правка  Вид  Терминал  Вкладки  Справка
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffc2a0 0xffffc2a0

B+>0x8049000 <_start>    mov    eax,0x4
      0x8049005 <_start+5>  mov    ebx,0x1
      0x804900a <_start+10> mov    ecx,0x804a000
      0x804900f <_start+15> mov    edx,0x8
      0x8049014 <_start+20> int     0x80
      0x804901b <_start+22> mov    eax,0x4

native process 8143 (asm) In: _start      L11  PC: 0x8049000
(gdb) layout regs
(gdb) █
```

Рисунок 0.24 (layout regs)

На предыдущих шагах была установлена точка останова по имени метки(_start).Про верьте это с помощью команды info breakpoints(кратко i b)

```
Файл  Правка  Вид  Терминал  Вкладки  Справка
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffc2a0 0xffffc2a0

B+>0x8049000 <_start>    mov    eax,0x4
      0x8049005 <_start+5>  mov    ebx,0x1
      0x804900a <_start+10> mov    ecx,0x804a000
      0x804900f <_start+15> mov    edx,0x8
      0x8049014 <_start+20> int     0x80
      0x804901b <_start+22> mov    eax,0x4

native process 8143 (asm) In: _start      L11  PC: 0x8049000
(gdb) layout regs
(gdb) info breakpoints
Num      Type      Disp Enb Address      What
1        breakpoint keep y  0x8049000 lab09-2.asm:11
breakpoint already hit 1 time
(gdb) █
```

Рисунок 0.25 (info breakpoints)

Установим еще одну точку останова по адресу инструкции. Адрес

инструкции можно увидеть в средней части экрана в левом столбце соответствующей инструкции. Определите адрес предпоследней инструкции (mov

ebx,0x0) и установим точку останова.

The screenshot shows a debugger window with a menu bar (Файл, Правка, Вид, Терминал, Вкладки, Справка). The main window is titled "Register group: general" and displays the following register values:

Register	Value
eax	0x0
ecx	0x0
edx	0x0
ebx	0x0
esp	0xffffc2a0

Below the register window, the assembly code is displayed:

```
0x804901b <_start+27> mov ebx,0x1
0x8049020 <_start+32> mov ecx,0x804a008
0x8049025 <_start+37> mov edx,0x7
0x804902a <_start+42> int 0x80
0x804902c <_start+44> mov eax,0x1
0x8049031 <_start+49> mov ebx,0x0
```

The status bar at the bottom indicates "native process 8809 (asm) In: _start L11 PC: 0x8049000". The command window shows the following commands and output:

```
(gdb) layout regs
(gdb) info breakpoints
Num Type Disp Enb Address What
1 breakpoint keep y 0x08049000 lab09-2.asm:11
breakpoint already hit 1 time
(gdb) break 0x8049031
```

Рисунок 0.26(установка точки останова)

Посмотрим информацию о всех установленных точках останова

The screenshot shows the same debugger window as Figure 0.26, but with two breakpoints set. The assembly code is the same:

```
0x804901b <_start+27> mov ebx,0x1
0x8049020 <_start+32> mov ecx,0x804a008
0x8049025 <_start+37> mov edx,0x7
0x804902a <_start+42> int 0x80
0x804902c <_start+44> mov eax,0x1
```

The status bar at the bottom indicates "native process 8809 (asm) In: _start L11 PC: 0x8049000". The command window shows the following commands and output:

```
(gdb) info breakpoints
Num Type Disp Enb Address What
1 breakpoint keep y 0x08049000 lab09-2.asm:11
breakpoint already hit 1 time
2 breakpoint keep y 0x08049031 lab09-2.asm:24
(gdb)
```

Рисунок 0.27(просмотр информации о установленных точках)

Посмотрим содержимое регистров также можно с помощью команды info registers

```
Терминал - vstomilova@dk3n55 - lab09
Файл Правка Вид Терминал Вкладки Справка
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

0x8049014 <_start+20> int 0x80
0x8049016 <_start+22> mov eax,0x4
0x804901b <_start+27> mov ebx,0x1
0x8049020 <_start+32> mov ecx,0x804a008
0x8049025 <_start+37> mov edx,0x7
0x804902a <_start+42> int 0x80
0x804902c <_start+44> mov eax,0x1

native process 8809 (asm) In: _start L11 PC: 0x8049000
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
--Type <RET> for more, q to quit, c to continue without paging--
```

Рисунок 0.28 (просмотр содержимого регистров)

Выполним 5 инструкций с помощью команды `stepi`(или `si`) и проследите за изменением значений регистров.

```
Терминал - vstomilova@dk3n55 - lab09
Файл  Правка  Вид  Терминал  Вкладки  Справка

Register group: general
eax      0x4      4
ecx      0x0      0
edx      0x0      0
ebx      0x1      1
esp      0xffffc2a0  0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

B+ 0x8049000 <_start>    mov    eax,0x4
0x8049005 <_start+5>    mov    ebx,0x1
>0x804900a <_start+10>   mov    ecx,0x804a000
0x804900f <_start+15>   mov    edx,0x8
0x8049014 <_start+20>   int    0x80
0x8049016 <_start+22>   mov    eax,0x4
0x804901b <_start+27>   mov    ebx,0x1

native process 8809 (asm) In: _start          L13    PC: 0x804900a
ebx      0x0      0
esp      0xffffc2a0  0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049005    0x8049005 <_start+5>
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) stepi
```

Рисунок 0.29(stepi)

```
Терминал - vstomilova@dk3n55 - lab09
Файл  Правка  Вид  Терминал  Вкладки  Справка

Register group: general
eax      0x4      4
ecx      0x0      0
edx      0x0      0
ebx      0x1      1
esp      0xffffc2a0  0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

B+ 0x8049000 <_start>    mov    eax,0x4
   0x8049005 <_start+5>   mov    ebx,0x1
>0x804900a <_start+10>   mov    ecx,0x804a000
   0x804900f <_start+15>   mov    edx,0x8
   0x8049014 <_start+20>   int    0x80
   0x8049016 <_start+22>   mov    eax,0x4
   0x804901b <_start+27>   mov    ebx,0x1

native process 8809 (asm) In: _start          L13    PC: 0x804900a
eax      0x4      4
ecx      0x0      0
edx      0x0      0
ebx      0x1      1
esp      0xffffc2a0  0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x804900a    0x804900a <_start+10>
--Type <RET> for more, q to quit, c to continue without paging--
```

Рисунок 0.30(info registers)

```
Терминал - vstomilova@dk3n55 - lab09
Файл Правка Вид Терминал Вкладки Справка
Register group: general
eax      0x4      4
ecx      0x804a000 134520832
edx      0x0      0
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

B+ 0x8049000 <_start>      mov     eax,0x4
    0x8049005 <_start+5>    mov     ebx,0x1
    0x804900a <_start+10>   mov     ecx,0x804a000
>0x804900f <_start+15>    mov     edx,0x8
    0x8049014 <_start+20>   int     0x80
    0x8049016 <_start+22>   mov     eax,0x4
    0x804901b <_start+27>   mov     ebx,0x1

native process 8809 (asm) In: _start      L14    PC: 0x804900f
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x804900a 0x804900a <_start+10>
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) stepi
(gdb)
```

Рисунок 0.31(stepi)


```
Терминал - vstomilova@dk3n55 - lab09
Файл Правка Вид Терминал Вкладки Справка

Register group: general
eax      0x4      4
ecx      0x804a000 134520832
edx      0x0      0
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

B+ 0x8049000 <_start>    mov    eax,0x4
    0x8049005 <_start+5>  mov    ebx,0x1
    0x804900a <_start+10> mov    ecx,0x804a000
>0x804900f <_start+15>  mov    edx,0x8
    0x8049014 <_start+20> int     0x80
    0x8049016 <_start+22> mov    eax,0x4
    0x804901b <_start+27> mov    ebx,0x1

native process 8809 (asm) In: _start L14 PC: 0x804900f
eax      0x4      4
ecx      0x804a000 134520832
edx      0x0      0
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x804900f 0x804900f <_start+15>
--Type <RET> for more, q to quit, c to continue without paging--
```

Рисунок 0.32(info registers)

```
Терминал - vstomilova@dk3n55 - lab09
Файл Правка Вид Терминал Вкладки Справка

Register group: general
eax      0x4      4
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

B+ 0x8049000 <_start>    mov     eax,0x4
    0x8049005 <_start+5>  mov     ebx,0x1
    0x804900a <_start+10> mov     ecx,0x804a000
    0x804900f <_start+15> mov     edx,0x8
>0x8049014 <_start+20>  int     0x80
    0x8049016 <_start+22> mov     eax,0x4
    0x804901b <_start+27> mov     ebx,0x1

native process 8809 (asm) In: _start L15 PC: 0x8049014
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x804900f 0x804900f <_start+15>
--Type <RET> for more, q to quit, c to continue without paging--
Quit
(gdb) stepi
(gdb)
```

Рисунок 0.33(step1)

```
Терминал - vstomilova@dk3n55 - lab09
Файл Правка Вид Терминал Вкладки Справка
Register group: general
eax      0x4      4
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

B+ 0x0049000 <_start>    mov    eax,0x4
    0x0049005 <_start+5>  mov    ebx,0x1
    0x004900a <_start+10> mov    ecx,0x804a000
    0x004900f <_start+15> mov    edx,0x8
>0x8049014 <_start+20>  int    0x80
    0x0049016 <_start+22> mov    eax,0x4
    0x004901b <_start+27> mov    ebx,0x1

native process 8809 (asm) In: _start L15 PC: 0x8049014
eax      0x4      4
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049014 0x8049014 <_start+20>
--Type <RET> for more, q to quit, c to continue without paging--
```

Рисунок 0.34(info registers)

```
Терминал - vstomilova@dk3n55 - lab09
Файл Правка Вид Терминал Вкладки Справка
Register group: general
eax      0x8      8
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

0x804900a <_start+10> mov ecx,0x804a000
0x804900f <_start+15> mov edx,0x8
0x8049014 <_start+20> int 0x80
>0x8049016 <_start+22> mov eax,0x4
0x804901b <_start+27> mov ebx,0x1
0x8049020 <_start+32> mov ecx,0x804a000
0x8049025 <_start+37> mov edx,0x7

native process 8809 (asm) In: _start L17 PC: 0x8049016
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049014 0x8049014 <_start+20>
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) stepi
(gdb)
```

Рисунок 0.35(stepi)

```
Терминал - vstomilova@dk3n55 - lab09
Файл  Правка  Вид  Терминал  Вкладки  Справка

Register group: general
eax      0x8      8
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

0x804900a <_start+10> mov ecx,0x804a000
0x804900f <_start+15> mov edx,0x8
0x8049014 <_start+20> int 0x80
>0x8049016 <_start+22> mov eax,0x4
0x804901b <_start+27> mov ebx,0x1
0x8049020 <_start+32> mov ecx,0x804a000
0x8049025 <_start+37> mov edx,0x7

native process 8809 (asm) In: _start L17 PC: 0x8049016
eax      0x8      8
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049016 0x8049016 <_start+22>
--Type <RET> for more, q to quit, c to continue without paging--
```

Рисунок 0.36(info registers)

```
Терминал - vstomilova@dk3n55 - lab09
Файл Правка Вид Терминал Вкладки Справка
Register group: general
eax      0x4      4
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

0x804900f <_start+15> mov     edx,0x8
0x8049014 <_start+20> int     0x80
0x8049016 <_start+22> mov     eax,0x4
>0x804901b <_start+27> mov     ebx,0x1
0x8049020 <_start+32> mov     ecx,0x804a000
0x8049025 <_start+37> mov     edx,0x7
0x804902a <_start+42> int     0x80

native process 8809 (asm) In: _start L18 PC: 0x804901b
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049016 0x8049016 <_start+22>
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) stepi
(gdb)
```

Рисунок 0.37(stepi)

```
Терминал - vstomilova@dk3n55 - lab09
Файл Правка Вид Терминал Вкладки Справка
Register group: general
eax      0x4      4
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

0x804900f <_start+15> mov     edx,0x8
0x8049014 <_start+20> int     0x80
0x8049016 <_start+22> mov     eax,0x4
>0x804901b <_start+27> mov     ebx,0x1
0x8049020 <_start+32> mov     ecx,0x804a000
0x8049025 <_start+37> mov     edx,0x7
0x804902a <_start+42> int     0x80

native process 8809 (asm) In: _start L18 PC: 0x804901b
eax      0x4      4
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x1      1
esp      0xffffc2a0 0xffffc2a0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x804901b 0x804901b <_start+27>
--Type <RET> for more, q to quit, c to continue without paging--
```

Рисунок 0.38(info registers)

При выполнении команды `stepi` (одной инструкции процессора) обычно изменяются:

1. **RIP/EIP** -указатель на следующую инструкцию (всегда меняется)
2. **RAX/EAX** -часто используется для вычислений и возврата значений
3. **RSP/ESP** -указатель стека (меняется при `push/pop`)
4. **Флаги** (ZF, CF, SF, OF) -после арифметических операций

Посмотрим значение переменной `msg1` по имени

```
(gdb) x/1sb &msg1
0x804a000 <msg1>:      "Hello, "
```

Рисунок 0.39(значение переменной `msg1`)

Посмотрим значение переменной `msg2` по адресу

```
(gdb) x/1sb 0x804a008
0x804a008 <msg2>:      "world!\n\034"
```

Рисунок 0.40(значение переменной `msg2`)

Изменим первый символ переменной `msg1`

```
(gdb) set {char}msg1='h'
'msg1' has unknown type; cast it to its declared type
(gdb) x/1sb &msg1
0x804a000 <msg1>:      "Hello, "
```

Рисунок 0.41(изменение первого символа `msg1`)

Заменяем любой символ во второй переменной msg2.

```
(gdb) set {char}&msg2='y'
(gdb) x/1sb &msg2
No symbol "msg2" in current context.
(gdb) x/1sb &msg2
0x804a008 <msg2>: "yorld!\n\034"
(gdb)
```

Рисунок 0.42 (изменение символа msg2)

Выведем в различных форматах (в шестнадцатеричном формате, двоичном формате и в символьном виде) значение регистра edx.

```
native process 6097 (asm) In: _start L11 PC: 0x80
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) set $ebx='2'
(gdb) p/s $ebx
$1 = 50
(gdb) p/t $ecx
$2 = 0
(gdb) p/s $edx
$3 = 0
(gdb) p/t $edx
$4 = 0
(gdb) p/x $edx
$5 = 0x0
(gdb)
```

Рисунок 0.43(значение регистра edx в разных форматах)

С помощью команды set изменим значение регистра ebx

```
(gdb) set $ebx='2'
(gdb) p/s $ebx
$6 = 50
(gdb) set $ebx=2
(gdb) p/s $ebx
$7 = 2
(gdb)
```

Рисунок 0.44 (изменение значения регистра ebx)

Завершим выполнение программы с помощью команды `continue` (сокращенно `c`) или `stepi` (сокращенно `si`) и выйдите из GDB с помощью команды `quit`(сокращенно `q`).

```
(gdb) c
Continuing.
hello, world!
[Inferior 1 (process 76534) exited normally]
(gdb)
```

Рисунок 0.45(завершение выполнения программы)

Обработка аргументов командной строки в GDB

Скопируем файл `lab8-2.asm`, созданный при выполнении лабораторной работы №8, с программой выводящей на экран аргументы командной строки в файл с именем `lab09-3.asm`

```
(gdb) layout asm
vstomilova@dk6n08 ~/work/arch-pc/lab09 $ cp ~/work/arch-pc/lab08/lab8-2.asm ~/work/arch-pc/
/lab09/lab09-3.asm
```

Рисунок 0.46(Создание копии файла)

Создадим исполняемый файл

```
vstomilova@dk6n08 ~/work/arch-pc/lab09 $ nasm -f elf -g -l lab09-3.lst lab09-3.asm
vstomilova@dk6n08 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-3 lab09-3.o
vstomilova@dk6n08 ~/work/arch-pc/lab09 $
```

Рисунок 0.47(создание исполняемого файла)

Загрузим исполняемый файл в отладчик, указав аргументы:

```
Терминал - vstomilova@dk6n08 - lab09
Файл Правка Вид Терминал Вкладки Справка
vstomilova@dk6n08 ~/work/arch-pc/lab09 $ gdb --args lab09-3 аргумент1 аргумент 2 'аргумент
3'
Exception caught while booting Guile.
Error in function "load-thunk-from-memory":
missing DT_GUILLE_ENTRY
gdb: warning: Could not complete Guile gdb module initialization from:
/usr/share/gdb/guile/gdb/boot.scm.
Limited Guile support is available.
Suggest passing --data-directory=/path/to/gdb/data-directory.
GNU gdb (Gentoo 16.3 vanilla) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://bugs.gentoo.org/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-3...
(gdb) 
```

Рисунок 0.48(загрузка исполняемого файла в отладчик)

Установим точку останова перед первой инструкцией в программе и запустим ее.

```
Reading symbols from lab09-3...
(gdb) b _start
Breakpoint 1 at 0x280490e8: file lab09-3.asm, line 5.
(gdb) 
```

Рисунок 0.49(установка точки перед первой инструкцией)

```
(gdb) run
Starting program: /afs/.dk.sci.pfu.edu.ru/home/v/s/vstomilova/work/arch-pc/lab09/lab09-3 а
ргумент1 аргумент 2 аргумент\ 3

Breakpoint 1, _start () at lab09-3.asm:5
5      pop ecx ; Извлекаем из стека в 'ecx' количество
(gdb) 
```

Рисунок 0.50(запуск)

Посмотрим остальные позиции стека—по адресу [esp+4] располагается адрес в памяти где находится имя программы, по адресу [esp+8] храниться адрес первого аргумента, по адресу [esp+12]—второго и т.д.

```

(gdb) x/x $esp
0xfffffc210:    0x00000005
(gdb) x/s *(void**)(esp + 4)
0xfffffc48f:    "/afs/.dk.sci.pfu.edu.ru/home/v/s/vstomilova/work/arch-pc/lab09/lab09-3"
(gdb) x/s *(void**)(esp + 8)
0xfffffc4d6:    "аргумент1"
(gdb) x/s *(void**)(esp + 12)
0xfffffc4e8:    "аргумент"
(gdb) x/s *(void**)(esp + 16)
0xfffffc4f9:    "2"
(gdb) x/s *(void**)(esp + 20)
0xfffffc4fb:    "аргумент 3"
(gdb) x/s *(void**)(esp + 24)
0x0:    <error: Cannot access memory at address 0x0>
(gdb)

```

Рисунок 0.51(Просмотр остальных позиций стека)

Каждый элемент этого массива — это адрес (указатель), который в 32-битной системе занимает 4 байта. Поэтому чтобы перейти, нужно прибавить 4 байта.

Задание для самостоятельной работы

Преобразуем программу из лабораторной работы №8 (Задание №1 для самостоятельной работы), реализовав вычисление значения функции $\square(\square)$ как подпрограмму.

```

vstomilova@dk6n08 ~/work/arch-pc/lab09 $ cp ~/work/arch-pc/lab08/lab8-4.asm ~/work/arch-pc/
/lab09/lab09-4.asm
vstomilova@dk6n08 ~/work/arch-pc/lab09 $

```

Рисунок 0.52 (создание копии файла)

Открыть ▾ + lab09-4.asm
~/work/arch-pc/lab09

```
1 %include 'in_out.asm'
2
3 SECTION .data
4 msg_func db "Функция: f(x) = 3(10+x)", 0
5 msg_result db "Результат: ", 0
6
7 SECTION .text
8 GLOBAL _start
9
10 _start:
11     mov eax, msg_func
12     call sprintf
13
14     pop ecx
15     pop edx
16     sub ecx, 1
17     mov esi, 0
18
19 next:
20     cmp ecx, 0h
21     jz _end
22     pop eax
23     call atoi
24
25     call _calculate_fx
26
27     add esi, eax
28     loop next
29
30 _end:
31     mov eax, msg_result
32     call sprintf
33     mov eax, esi
34     call iprintLF
35     call quit
36
37 _calculate_fx:
38     add eax, 10
39     mov ebx, 3
40     mul ebx
41     ret
```

Рисунок 0.53(текст программы)

```

vstomilova@dk6n08 ~/work/arch-pc/lab09 $ gedit lab09-4.asm
vstomilova@dk6n08 ~/work/arch-pc/lab09 $ nasm -f elf lab09-4.asm
vstomilova@dk6n08 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-4 lab09-4.o
vstomilova@dk6n08 ~/work/arch-pc/lab09 $ ./lab09-4 3 0 1 2
Функция:  $f(x) = 3(10+x)$ 
Результат: 138
vstomilova@dk6n08 ~/work/arch-pc/lab09 $ ./lab09-4 2 3 4
Функция:  $f(x) = 3(10+x)$ 
Результат: 117
vstomilova@dk6n08 ~/work/arch-pc/lab09 $

```

Рисунок 0.54(проверка работы программы)

В листинге приведена программа вычисления выражения $(3+2)*4+5$. При запуске данная программа дает неверный результат. Проверьте это. С помощью отладчика GDB, анализируя изменения значений регистров, определите ошибку и исправьте ее.

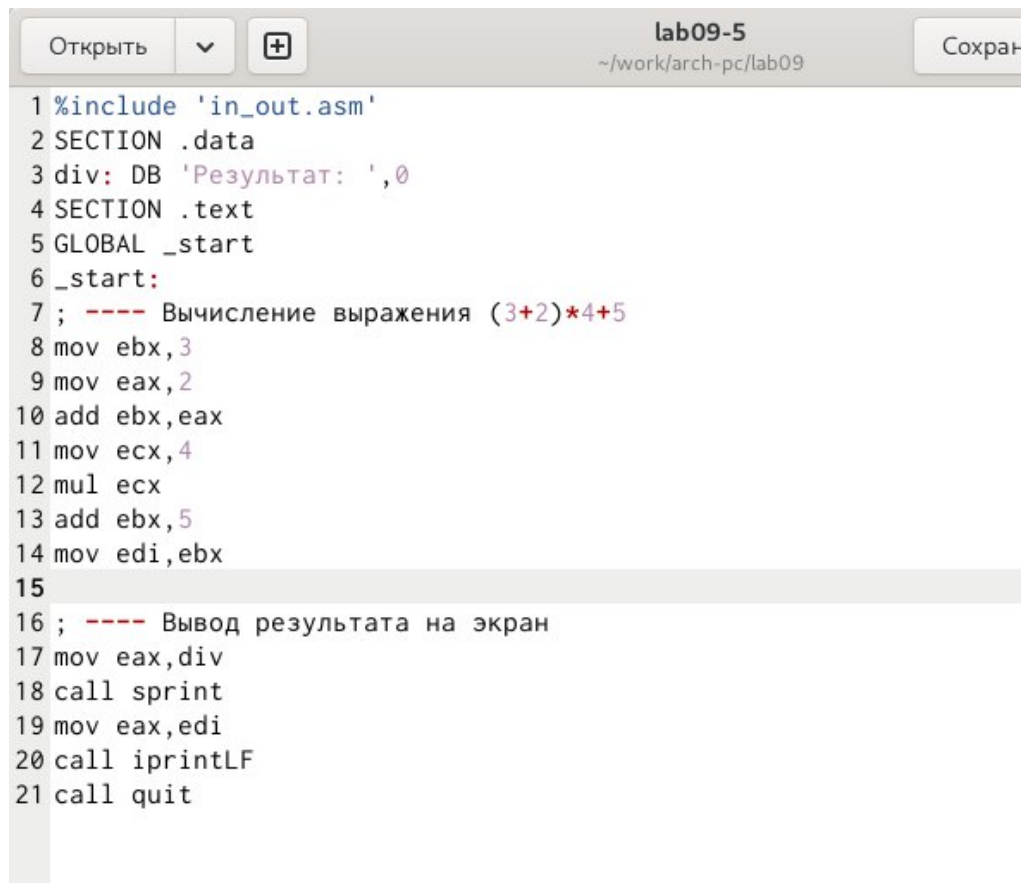
Создаю новый файл lab09-5 и открываю его при помощи команды gedit.

```

vstomilova@dk6n08 ~/work/arch-pc/lab09 $ touch lab09-5
vstomilova@dk6n08 ~/work/arch-pc/lab09 $ gedit lab09-5
vstomilova@dk6n08 ~/work/arch-pc/lab09 $

```

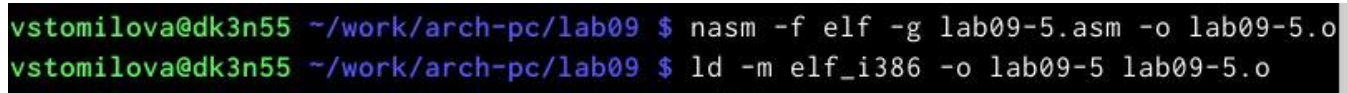
Рисунок 0.55(создание и открытие файла)



```
1 %include 'in_out.asm'
2 SECTION .data
3 div: DB 'Результат: ',0
4 SECTION .text
5 GLOBAL _start
6 _start:
7 ; ---- Вычисление выражения (3+2)*4+5
8 mov ebx,3
9 mov eax,2
10 add ebx,eax
11 mov ecx,4
12 mul ecx
13 add ebx,5
14 mov edi,ebx
15
16 ; ---- Вывод результата на экран
17 mov eax,div
18 call sprint
19 mov eax,edi
20 call iprintLF
21 call quit
```

Рисунок 0.56(текст программы)

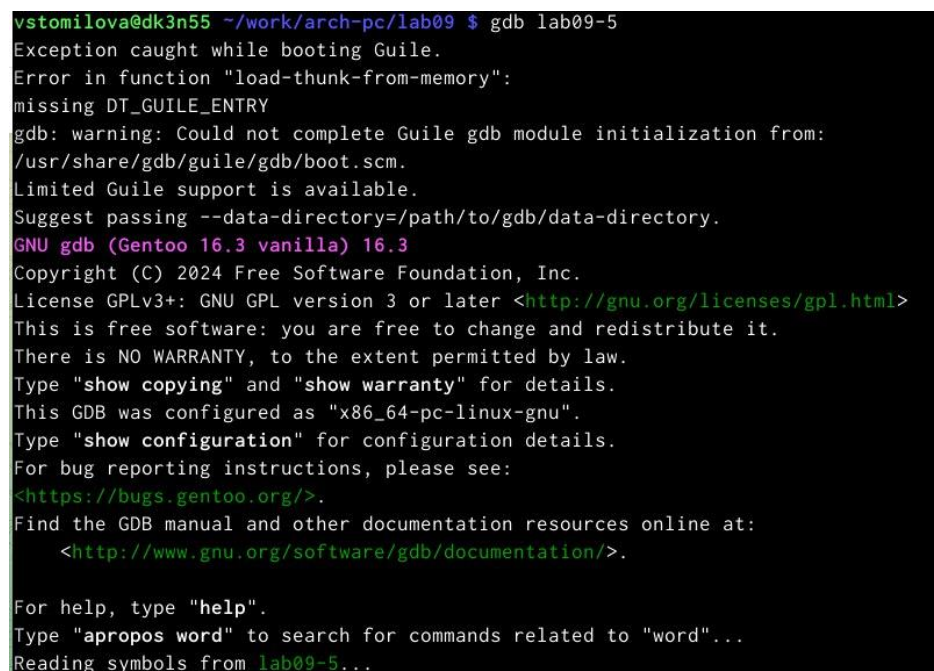
Создаю исполняемый файл.



```
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ nasm -f elf -g lab09-5.asm -o lab09-5.o
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-5 lab09-5.o
```

Рисунок 0.57(создание исполняемого файла)

Загружаю исполняемый файл в отладчик командой gdb.



```
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ gdb lab09-5
Exception caught while booting Guile.
Error in function "load-thunk-from-memory":
missing DT_GUILLE_ENTRY
gdb: warning: Could not complete Guile gdb module initialization from:
/usr/share/gdb/guile/gdb/boot.scm.
Limited Guile support is available.
Suggest passing --data-directory=/path/to/gdb/data-directory.
GNU gdb (Gentoo 16.3 vanilla) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB is configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://bugs.gentoo.org/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-5...
```

Рисунок 0.58(загрузка исполняемого файла в отладчик)

Устанавливаю брейкпоинт.

```
(gdb) break _start
Breakpoint 1 at 0x80490e8: file lab09-5.asm, line 8.
```

Рисунок 0.59(установка брейкпоинта)

Запускаю программу.

```
(gdb) run
Starting program: /afs/.dk.sci.pfu.edu.ru/home/v/s/vstomilova/work/arch-pc/lab09-5
/lab09-5

Breakpoint 1, _start () at lab09-5.asm:8
8      mov ebx, 3
```

Рисунок 0.60(запуск программы)

```
(gdb) print $eax
$1 = 0
(gdb) continue
Continuing.
Результат: 10
[Inferior 1 (process 3719) exited normally]
```

Рисунок 0.61(Результат до исправления кода)

Ищу ошибку.


```
Терминал - vstomilova@dk3n55 - lab09
Файл Правка Вид Терминал Вкладки Справка
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffc290 0xffffc290
ebp      0x0      0x0

B+>0x80490e8 <_start>    mov     ebx,0x3
    0x80490ed <_start+5>  mov     eax,0x2
    0x80490f2 <_start+10> add     ebx,eax
    0x80490f4 <_start+12> mov     ecx,0x4
    0x80490f9 <_start+17> mul     ecx
    0x80490fb <_start+19> add     ebx,0x5

native process 4304 (asm) In: _start      L8      PC:
(gdb) layout regs
(gdb) □
```

Рисунок 0.62(layout regs)

```
Терминал - vstomilova@dk3n55 - lab09
Файл Правка Вид Терминал Вкладки Справка
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x3      3
esp      0xffffc290 0xffffc290
ebp      0x0      0x0

B+ 0x80490e8 <_start>    mov     ebx,0x3
>0x80490ed <_start+5>  mov     eax,0x2
    0x80490f2 <_start+10> add     ebx,eax
    0x80490f4 <_start+12> mov     ecx,0x4
    0x80490f9 <_start+17> mul     ecx
    0x80490fb <_start+19> add     ebx,0x5

native process 4304 (asm) In: _start      L9      PC
(gdb) layout regs
(gdb) stepi
(gdb) □
```

Рисунок 0.63(первый stepi)

```
Терминал - vstomilova@dk3n55 - lab09

Файл  Правка  Вид  Терминал  Вкладки  Справка

Register group: general
eax      0x2      2
ecx      0x0      0
edx      0x0      0
ebx      0x3      3
esp      0xffffc290  0xffffc290
ebp      0x0      0x0

B+ 0x80490e8 <_start>    mov     ebx,0x3
    0x80490ed <_start+5>  mov     eax,0x2
>0x80490f2 <_start+10>   add     ebx,eax
    0x80490f4 <_start+12>  mov     ecx,0x4
    0x80490f9 <_start+17>  mul     ecx
    0x80490fb <_start+19>  add     ebx,0x5

native process 4304 (asm) In: _start      L10  PC
(gdb) layout regs
(gdb) stepi
(gdb) stepi
(gdb) █
```

Рисунок 0.64(второй stepi)

```
Терминал - vstomilova@dk3n55 - lab09

Файл  Правка  Вид  Терминал  Вкладки  Справка

Register group: general
eax      0x2      2
ecx      0x0      0
edx      0x0      0
ebx      0x5      5
esp      0xffffc290  0xffffc290
ebp      0x0      0x0

B+ 0x80490e8 <_start>    mov     ebx,0x3
    0x80490ed <_start+5>  mov     eax,0x2
    0x80490f2 <_start+10>  add     ebx,eax
>0x80490f4 <_start+12>   mov     ecx,0x4
    0x80490f9 <_start+17>  mul     ecx
    0x80490fb <_start+19>  add     ebx,0x5

native process 4304 (asm) In: _start      L11  PC
(gdb) layout regs
(gdb) stepi
(gdb) stepi
(gdb) stepi
(gdb) █
```

Рисунок 0.65(третий stepi)

```
Терминал - vstomilova@dk3n55 - lab09
Файл Правка Вид Терминал Вкладки Справка
Register group: general
eax      0x8      8
ecx      0x4      4
edx      0x0      0
ebx      0x5      5
esp      0xffffc290 0xffffc290
ebp      0x0      0x0

0x80490f2 <_start+10> add ebx,eax
0x80490f4 <_start+12> mov ecx,0x4
0x80490f9 <_start+17> mul ecx
>0x80490fb <_start+19> add ebx,0x5
0x80490fe <_start+22> mov edi,ebx
0x8049100 <_start+24> mov eax,0x804a000

native process 4304 (asm) In: _start L13 F
(gdb) layout regs
(gdb) stepi
(gdb) stepi
(gdb) stepi
(gdb) stepi
(gdb) stepi
(gdb) stepi
(gdb) 
```

Рисунок 0.66(stepі находим ошибку)

```
lab09-5.asm
~/work/arch-pc/lab09

1 %include 'in_out.asm'
2
3 SECTION .data
4 div: DB 'Результат: ',0
5
6 SECTION .text
7 GLOBAL _start
8 _start:
9
10 mov ebx,3
11 mov eax,2
12 add ebx,eax
13 mov ecx,ebx
14 mov ecx,4
15 mul ecx
16 add eax,5
17 mov edi,eax
18
19 mov eax,div
20 call sprint
21 mov eax,edi
22 call iprintLF
23
24 call quit
```

Рисунок 0.67(исправленный код)

```
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ gedit lab09-5.asm
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ nasm -f elf lab09-5.asm
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-5 lab09-5.o
vstomilova@dk3n55 ~/work/arch-pc/lab09 $ ./lab09-5
Результат: 13
vstomilova@dk3n55 ~/work/arch-pc/lab09 $
```

Рисунок 0.68(результат после исправления)

5 Выводы

Я приобрела навыки написания программ с использованием подпрограмм. Ознакомилась с методами отладки при помощи GDB и его основными возможностями

Список литературы

1. GDB:TheGNUProjectDebugger.—URL:<https://www.gnu.org/software/gdb/>.
2. GNUBashManual.—2016.—URL:<https://www.gnu.org/software/bash/manual/>.
3. Midnight CommanderDevelopment Center.—2021.—URL: <https://midnight-commander.org/>.
4. NASMAssemblyLanguageTutorials.—2021.—URL: <https://asmtutor.com/>.
5. Newham C. Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 с. —(In a Nutshell). —ISBN 0596009658.—URL: [http://www.amazon.com/Learning bash-Shell-Programming-Nutshell/dp/0596009658](http://www.amazon.com/Learning-bash-Shell-Programming-Nutshell/dp/0596009658).
6. RobbinsA. Bash Pocket Reference.—O'Reilly Media,2016.—156 с.—ISBN 978-1491941591.
7. TheNASMdocumentation.—2021.—URL:<https://www.nasm.us/docs.php>.
8. Zarrelli G. Mastering Bash.—Packt Publishing,2017.—502 с.—ISBN 9781784396879.
9. Колдаев В. Д.,Лупин С. А.Архитектура ЭВМ.—М. : Форум,2018.
10. Куляс О. Л.,Никитин К. А. Курс программирования наASSEMBLER.— М. : Солон-Пресс, 2017.
11. НовожиловО.П.Архитектура ЭВМисистем.—М.:Юрайт,2016.
12. Расширенныйассемблер:NASM.—2021.—URL:<https://www.opennet.ru/docs/RUS/nasm/>.
13. Робачевский А.,Немнюгин С.,Стесик О. Операционная система UNIX.— 2-е изд.—БХВ Петербург, 2010.—656 с.—ISBN 978-5-94157-538-1.
14. СтоляроваА.Программированияязыкеассемблера NASMдляОСUnix.—2-еизд.— М. : МАКСПресс,2011.—URL:http://www.stolyarov.info/books/asm_unix.
15. Таненбаум Э. Архитектура компьютера. — 6-е изд. — СПб. : Питер, 2013. — 874 с. — (Классика Computer Science).
16. Таненбаум Э.,Бос Х. Современные операционные системы.—4-е изд.— СПб.: Питер, 2015. —1120 с.—(Классика Computer Science)